

Rapport de projet

Gomoku

Intelligence artificielle pour un jeu stratégique à deux joueurs

Auteurs : LI Wenxiao

REN Ziyi

Responsable de l'UE : Elise Bonzon

Année universitaire : 2025–2026

Licence 3 – Mathématiques et Informatique

UE Intelligence Artificielle

Table des matières

1	Introduction	2
2	Présentation du jeu	2
2.1	<i>Règles du Gomoku</i>	2
2.2	<i>Choix du plateau 15×15 et hypothèses</i>	2
3	Moteur de jeu	3
3.1	<i>Représentation de l'état</i>	3
3.2	<i>Gestion des coups et détection de victoire</i>	3
3.3	<i>Validation du moteur</i>	3
4	Architecture des intelligences artificielles	4
4.1	<i>Principe général : recherche, évaluation et coups candidats</i>	4
4.2	<i>Génération des coups candidats</i>	4
4.3	<i>Niveaux de difficulté</i>	4
5	Fonctions d'évaluation	5
5.1	<i>Principe général du score</i>	5
5.2	<i>Évaluation A : alignements consécutifs</i>	5
5.3	<i>Évaluation B : motifs ouverts, fermés et menaces combinées</i>	6
5.4	<i>Évaluation C : fenêtres de cinq cases</i>	7
6	Algorithmes de recherche	8
6.1	<i>Minimax</i>	8
6.2	<i>Élagage alpha-bêta</i>	9
6.3	<i>Ordonnancement des coups</i>	10
7	Expériences	10
7.1	<i>Expérience 1 : efficacité de l'élagage alpha-bêta</i>	10
7.2	<i>Expérience 2 : comparaison des fonctions d'évaluation</i>	12
7.3	<i>Expérience 3 : sélection des configurations d'IA</i>	14
8	Tournoi entre intelligences artificielles	16
8.1	<i>Protocole expérimental</i>	16
8.2	<i>Résultats expérimentaux du tournoi</i>	16
8.3	<i>Observations sur les résultats</i>	19
9	Difficultés rencontrées	20
10	Conclusion	21
11	Utilisation de l'IA générative	22
A	Annexes	22

1 Introduction

Ce projet a été réalisé dans le cadre de l'UE Intelligence Artificielle. Il consiste à implémenter un jeu stratégique à deux joueurs, puis à développer des intelligences artificielles capables d'y jouer.

Nous avons choisi le Gomoku, aussi appelé cinq en ligne. C'est un jeu déterministe à information parfaite : les deux joueurs connaissent tout l'état du plateau, et aucune règle ne dépend du hasard. Ce cadre se prête donc bien aux algorithmes de recherche vus en cours, notamment Minimax et l'élagage alpha-bêta.

Le travail réalisé comprend plusieurs parties : un moteur de jeu, plusieurs niveaux d'intelligence artificielle, trois fonctions d'évaluation et des expériences pour comparer les performances obtenues. Le rapport présente d'abord les règles du jeu et les choix d'implémentation du moteur, puis l'architecture des IA, les fonctions d'évaluation, les algorithmes de recherche, les expériences, le tournoi entre IA et les difficultés rencontrées.

2 Présentation du jeu

2.1 Règles du Gomoku

Le Gomoku est un jeu de stratégie à deux joueurs. Les joueurs jouent chacun leur tour en plaçant un pion sur une case vide du plateau. Le but est d'être le premier à aligner cinq pions consécutifs de sa couleur.

Un alignement peut être :

- horizontal ;
- vertical ;
- diagonal.

Une partie se termine lorsqu'un joueur forme une ligne de cinq pions ou plus. Si le plateau est plein sans qu'aucun joueur n'ait gagné, la partie est considérée comme nulle.

Dans notre programme, les cases du plateau sont représentées par des entiers : 0 pour une case vide, 1 pour le premier joueur et -1 pour le second joueur. Lors de l'affichage dans le terminal, ces valeurs sont converties en symboles : . pour une case vide, X pour le joueur 1 et O pour le joueur -1 .

2.2 Choix du plateau 15×15 et hypothèses

Nous utilisons un plateau de taille 15×15 . Cette taille est classique pour le Gomoku et rend le jeu suffisamment intéressant du point de vue stratégique, tout en restant raisonnable pour un projet d'implémentation.

Nous avons choisi une version simplifiée des règles :

- les règles professionnelles comme l'interdiction du double-trois ou du double-quatre ne sont pas prises en compte ;
- une ligne de cinq pions ou plus est considérée comme gagnante ;
- les deux joueurs jouent simplement à tour de rôle sur des cases vides ;
- il n'y a pas de règle d'échange ni de restriction particulière pour le premier joueur.

Ces choix permettent de garder des règles simples et claires, afin de se concentrer principalement sur les aspects liés à l'intelligence artificielle : représentation des états, génération des coups, recherche dans l'arbre de jeu, fonctions d'évaluation et analyse expérimentale.

3 Moteur de jeu

3.1 Représentation de l'état

L'état du jeu est représenté par une matrice de taille 15×15 , stockée dans l'attribut `Board.grid`. En Python, cette structure correspond à une liste de listes d'entiers :

```
self.grid: List[List[int]]
```

Chaque case du plateau peut prendre trois valeurs :

Valeur interne	Signification	Affichage terminal
0	Case vide	.
1	Joueur 1	X
-1	Joueur 2	O

TABLE 1 – Représentation interne et affichage du plateau

Cette représentation numérique facilite les calculs effectués par le moteur de jeu et par les fonctions d'évaluation. L'affichage en caractères ., X et O est uniquement utilisé pour rendre le plateau lisible dans le terminal.

Le choix des valeurs 1 et -1 est également pratique pour alterner les joueurs : le joueur suivant peut être obtenu en changeant simplement le signe du joueur courant. Dans les algorithmes de recherche, le joueur MAX n'est pas forcément le joueur 1 ; il correspond au joueur pour lequel l'IA cherche un coup.

3.2 Gestion des coups et détection de victoire

Un coup est légal si les coordonnées choisies appartiennent au plateau et si la case est vide. Dans notre représentation, cela signifie que la case correspondante dans `Board.grid` doit contenir la valeur 0.

Lorsqu'un coup est joué, la valeur du joueur courant est placée dans la grille :

- 1 pour le joueur affiché par X ;
- -1 pour le joueur affiché par O.

Après chaque coup, le moteur vérifie si le joueur vient de former un alignement de cinq pions ou plus. La vérification se fait dans quatre directions :

- horizontale ;
- verticale ;
- diagonale principale ;
- diagonale secondaire.

À partir du dernier coup joué, on compte les pions identiques dans les deux sens de chaque direction. Si le total atteint au moins cinq, le joueur gagne la partie.

Cette méthode est plus efficace qu'un parcours complet du plateau après chaque coup, car seule la zone liée au dernier coup joué doit être examinée.

3.3 Validation du moteur

Pour vérifier la correction du moteur, nous avons testé plusieurs situations caractéristiques : victoires horizontales, verticales, diagonales, coups illégaux et matchs nuls.

Ces tests permettent de vérifier que les règles principales du Gomoku sont bien respectées par notre moteur.

Cas testé	Résultat attendu	Résultat obtenu
Cinq pions horizontaux	Victoire	Correct
Cinq pions verticaux	Victoire	Correct
Cinq pions diagonaux	Victoire	Correct
Coup sur une case occupée	Coup refusé	Correct
Plateau plein sans gagnant	Match nul	Correct

TABLE 2 – Exemples de validation du moteur de jeu

4 Architecture des intelligences artificielles

4.1 Principe général : recherche, évaluation et coups candidats

Les intelligences artificielles reposent toutes sur la même structure générale. Pour choisir un coup, une IA combine trois éléments :

- une génération de coups candidats ;
- un algorithme de recherche ;
- une fonction d'évaluation.

La génération des coups candidats détermine les coups qui seront examinés. L'algorithme de recherche explore ensuite les conséquences possibles de ces coups. Enfin, la fonction d'évaluation attribue un score aux positions atteintes lorsque la profondeur maximale est atteinte ou lorsqu'un état terminal est rencontré.

Dans les algorithmes de recherche, on se place du point de vue du joueur qui doit choisir un coup. Ce joueur est appelé MAX, tandis que son adversaire est appelé MIN. Ainsi, un score positif indique une position favorable au joueur qui lance la recherche, et un score négatif indique une position favorable à son adversaire.

De manière générale, les fonctions d'évaluation suivent le principe suivant :

$$score = avantage(MAX) - avantage(MIN)$$

Cette convention permet d'utiliser les mêmes fonctions d'évaluation quel que soit le joueur qui appelle l'IA.

4.2 Génération des coups candidats

Une génération naïve des coups consisterait à considérer toutes les cases vides du plateau. Cependant, sur un plateau 15×15 , cette méthode produit un facteur de branchement très élevé, surtout au début de la partie.

Pour réduire le nombre de positions explorées, nous limitons les coups candidats aux cases proches des pions déjà placés. Cette stratégie repose sur l'idée qu'un coup joué loin de tous les autres pions a rarement un impact stratégique immédiat.

Dans notre implémentation, les coups candidats sont donc principalement les cases vides situées dans le voisinage des pions existants. Sur un plateau vide, le premier coup candidat est le centre du plateau. Cette optimisation réduit le nombre de coups à analyser tout en conservant la majorité des coups stratégiquement pertinents.

4.3 Niveaux de difficulté

Nous avons défini trois niveaux de difficulté : Easy, Medium et Hard. Ces niveaux ne sont pas choisis uniquement de manière théorique. Ils ont été déterminés à partir des

résultats expérimentaux, en tenant compte de plusieurs critères : le temps de calcul, le nombre de noeuds explorés, la qualité des coups choisis et les résultats en matchs entre IA.

Les détails de ces comparaisons sont présentés dans la section 7. La configuration finale retenue est la suivante :

Difficulté	Profondeur de recherche	Fonction d'évaluation
Easy	1	Évaluation A
Medium	3	Évaluation A
Hard	2	Évaluation B

TABLE 3 – Configuration finale des niveaux de difficulté

Le niveau Easy correspond à une IA rapide, avec une recherche très limitée. Le niveau Medium utilise la même fonction d'évaluation que Easy, mais avec une profondeur plus grande. Il peut donc anticiper davantage les réponses de l'adversaire, tout en gardant une fonction d'évaluation relativement simple.

Le niveau Hard utilise une fonction d'évaluation plus précise des menaces, avec une profondeur de recherche plus faible que Medium afin de conserver un temps de calcul acceptable. Ce choix reflète le compromis observé dans les expériences : une évaluation plus riche peut parfois être plus utile qu'une profondeur plus élevée, mais elle augmente aussi le coût de calcul.

5 Fonctions d'évaluation

5.1 Principe général du score

Les fonctions d'évaluation servent à estimer la qualité d'une position lorsque la recherche ne peut pas continuer jusqu'à la fin réelle de la partie. Elles sont donc essentielles pour Minimax et alpha-bêta.

Un état gagnant pour MAX reçoit un score très élevé. Un état gagnant pour MIN reçoit un score très faible. Pour les positions non terminales, le score dépend des motifs présents sur le plateau.

L'objectif est d'attribuer un score plus élevé aux positions qui offrent de bonnes possibilités d'attaque pour MAX, tout en pénalisant les positions dans lesquelles MIN possède des menaces importantes.

5.2 Évaluation A : alignements consécutifs

L'Évaluation A est la plus simple. Elle attribue un score selon la longueur des alignements consécutifs déjà présents sur le plateau.

Elle prend par exemple en compte les motifs suivants :

XX
XXX
XXXX

Un exemple de pondération est donné dans le tableau suivant.

Cette évaluation est rapide à calculer. Elle permet de détecter les alignements déjà formés, mais elle ne distingue pas les alignements ouverts des alignements bloqués. Par

Motif	Score
Deux pions consécutifs	10
Trois pions consécutifs	100
Quatre pions consécutifs	1000
Cinq pions consécutifs	100000

TABLE 4 – Exemple de pondération pour l'Évaluation A

exemple, un alignement de trois pions avec deux extrémités libres est beaucoup plus dangereux qu'un alignement bloqué, mais l'Évaluation A peut leur donner une valeur similaire.

5.3 Évaluation B : motifs ouverts, fermés et menaces combinées

L'Évaluation B améliore l'Évaluation A en tenant compte des extrémités des alignements. Un alignement ouvert est plus dangereux qu'un alignement bloqué, car il peut être prolongé plus facilement. Cette évaluation distingue donc les motifs ouverts, qui possèdent deux extrémités libres, et les motifs fermés, qui ne peuvent être prolongés que d'un seul côté.

Exemples de motifs :

```
_XXX_ : trois ouvert
OXXX_ : trois fermé
_XXXX_ : quatre ouvert
OXXXX_ : quatre fermé
```

Un trois ouvert possède deux extrémités libres et peut donc évoluer vers une menace plus forte. Un trois fermé est moins dangereux, car il ne peut être prolongé que d'un seul côté. De même, un quatre ouvert représente une menace très forte, car il offre plusieurs possibilités de gagner au coup suivant.

Motif	Exemple	Score
Trois ouvert	_XXX_	500
Trois fermé	OXXX_	100
Quatre ouvert	_XXXX_	10000
Quatre fermé	OXXXX_	1000

TABLE 5 – Exemples de pondération des motifs simples pour l'Évaluation B

Dans une première version, l'Évaluation B ne prenait en compte que ces motifs simples. Cependant, les tests ont montré que cela n'était pas suffisant pour bien évaluer certaines positions de Gomoku. En effet, une position peut être dangereuse non seulement parce qu'elle contient un alignement fort, mais aussi parce qu'elle crée plusieurs menaces simultanément. Ces menaces combinées sont particulièrement importantes, car l'adversaire ne peut souvent en bloquer qu'une seule au coup suivant.

Nous avons donc ajouté une fonction spécifique, `score_open_segment_combinations`, afin d'attribuer des bonus supplémentaires aux combinaisons de menaces. Cette amélioration permet à l'IA de ne plus seulement regarder la force d'une ligne isolée, mais aussi la structure globale des menaces présentes sur le plateau.

Combinaison détectée	Bonus ajouté
Double trois ouvert	+6000
Trois ouvert + quatre fermé	+12000
Trois ouvert + quatre ouvert	+18000
Double quatre fermé	+8000
Trois ouvert + trois endormi	+1200

TABLE 6 – Bonus ajoutés pour les menaces combinées dans l'Évaluation B

Par exemple, un double trois ouvert peut forcer l'adversaire à choisir entre deux défenses, ce qui donne souvent une attaque décisive au joueur courant. De même, la combinaison d'un trois ouvert et d'un quatre fermé est très forte, car elle associe une menace immédiate à une possibilité de développement. Ces motifs composés sont donc mieux valorisés dans la nouvelle version de l'Évaluation B.

Enfin, nous avons aussi légèrement renforcé l'aspect défensif de cette évaluation. La première version calculait le score de la manière suivante :

```
return my_score - opp_score
```

Dans la version finale, le score devient :

```
return my_score - 1.08 * opp_score
```

Le score de l'adversaire est donc multiplié par 1.08, ce qui signifie que les menaces adverses sont considérées comme légèrement plus importantes que dans la version précédente. Ce choix rend l'IA plus prudente : elle bloque plus facilement les menaces dangereuses de l'adversaire et évite davantage de laisser apparaître des situations perdantes.

Ainsi, l'Évaluation B reste plus coûteuse que l'Évaluation A, mais elle reconnaît mieux les menaces directes, les opportunités d'attaque et les positions nécessitant une défense immédiate. Elle constitue donc un compromis entre simplicité de calcul et qualité stratégique.

5.4 Évaluation C : fenêtres de cinq cases

L'Évaluation C analyse toutes les fenêtres de cinq cases possibles dans les directions horizontale, verticale et diagonale. Pour chaque fenêtre, on compte :

- le nombre de pions de MAX ;
- le nombre de pions de MIN ;
- le nombre de cases vides.

Une fenêtre est considérée comme utile si elle contient les pions d'un seul joueur et des cases vides. Si elle contient à la fois des pions de MAX et des pions de MIN, elle est bloquée et sa valeur est nulle.

Exemples :

```
XX_X_ : potentiel pour MAX
XXX__ : potentiel pour MAX
XX0__ : fenêtre bloquée, score nul
```

Cette évaluation ne se limite pas aux alignements déjà consécutifs. Elle estime aussi le potentiel d'une position à l'intérieur de fenêtres de taille cinq, ce qui est naturel pour le Gomoku puisque l'objectif est précisément de former une ligne de cinq pions.

L'Évaluation C est plus coûteuse à calculer que les deux précédentes, mais elle donne une estimation plus complète de la position.

6 Algorithmes de recherche

6.1 Minimax

L'algorithme Minimax s'applique aux jeux à deux joueurs, déterministes et à somme constante. Le principe est de représenter les coups possibles sous forme d'un arbre de recherche. À chaque niveau de l'arbre, un des deux joueurs choisit un coup.

On se place du point de vue du joueur qui doit choisir un coup. Ce joueur est appelé MAX, car il cherche à maximiser le score obtenu. Son adversaire est appelé MIN, car il cherche à minimiser ce même score. Dans notre implémentation, MAX n'est donc pas toujours le joueur noir : MAX correspond au joueur pour lequel l'IA lance la recherche.

Comme l'arbre complet du Gomoku est trop grand pour être exploré jusqu'aux états terminaux, nous utilisons une profondeur maximale. Lorsque cette profondeur est atteinte, une fonction d'évaluation estime la valeur de la position.

Les conditions d'arrêt sont :

- la profondeur maximale est atteinte ;
- la position est terminale ;
- aucun coup candidat n'est disponible.

Algorithme 1 : Minimax à profondeur limitée

Input : Plateau B , profondeur d , joueur courant p , joueur MAX r

Output : Un couple $(score, coup)$

```

if  $d = 0$  ou  $B$  est terminal then
   $\_$  return  $(Eval(B, r), \emptyset)$ 
 $C \leftarrow Actions(B)$ 
if  $C$  est vide then
   $\_$  return  $(Eval(B, r), \emptyset)$ 

if  $p = r$  then
   $v \leftarrow -\infty$ ;  $best \leftarrow \emptyset$ 
  foreach  $a \in C$  do
    appliquer  $a$ 
     $(s, \_ ) \leftarrow Minimax(B, d - 1, -p, r)$ 
    annuler  $a$ 
    if  $s > v$  then
       $\_$   $v \leftarrow s$ ;  $best \leftarrow a$ 
   $\_$ 
else
   $v \leftarrow +\infty$ ;  $best \leftarrow \emptyset$ 
  foreach  $a \in C$  do
    appliquer  $a$ 
     $(s, \_ ) \leftarrow Minimax(B, d - 1, -p, r)$ 
    annuler  $a$ 
    if  $s < v$  then
       $\_$   $v \leftarrow s$ ;  $best \leftarrow a$ 
   $\_$ 
return  $(v, best)$ 

```

Dans le programme, l'algorithme retourne à la fois le score obtenu et le meilleur coup trouvé. Cela permet à l'IA de jouer directement le coup associé à la meilleure valeur

Minimax.

6.2 Élagage alpha-bêta

L'élagage alpha-bêta est une amélioration de Minimax. Il garde le même principe, mais évite d'explorer certaines branches qui ne peuvent plus changer la décision finale.

On maintient deux valeurs :

- α : la meilleure valeur déjà trouvée pour MAX ;
- β : la meilleure valeur déjà trouvée pour MIN.

Si, pendant la recherche, on obtient $\alpha \geq \beta$, on peut arrêter l'exploration de la branche courante.

Algorithme 2 : Minimax avec élagage alpha-bêta

Input : Plateau B , profondeur d , bornes α, β , joueur courant p , joueur MAX r

Output : Un couple ($score, coup$)

if $d = 0$ ou B est terminal **then**

return ($Eval(B, r), \emptyset$)

$C \leftarrow Actions(B)$

if C est vide **then**

return ($Eval(B, r), \emptyset$)

if $p = r$ **then**

$v \leftarrow -\infty$; $best \leftarrow \emptyset$

foreach $a \in C$ **do**

 appliquer a

$(s, _) \leftarrow AlphaBeta(B, d - 1, \alpha, \beta, -p, r)$

 annuler a

if $s > v$ **then**

$v \leftarrow s$; $best \leftarrow a$

$\alpha \leftarrow \max(\alpha, v)$

if $\alpha \geq \beta$ **then**

 arrêter la boucle

else

$v \leftarrow +\infty$; $best \leftarrow \emptyset$

foreach $a \in C$ **do**

 appliquer a

$(s, _) \leftarrow AlphaBeta(B, d - 1, \alpha, \beta, -p, r)$

 annuler a

if $s < v$ **then**

$v \leftarrow s$; $best \leftarrow a$

$\beta \leftarrow \min(\beta, v)$

if $\alpha \geq \beta$ **then**

 arrêter la boucle

return ($v, best$)

Dans nos expériences, nous mesurons notamment le nombre de noeuds évalués, le nombre de coupures, la profondeur atteinte et le temps de calcul. Ces mesures permettent de comparer Minimax classique et Minimax avec élagage alpha-bêta.

6.3 Ordonnancement des coups

L'ordre dans lequel les fils sont visités est important pour l'efficacité de l'élagage alpha-bêta. Si un bon coup est trouvé tôt, les bornes α et β se resserrent plus vite, ce qui permet d'élaguer davantage de branches.

Dans notre cas, nous ne considérons pas toutes les cases vides du plateau. Les actions explorées sont des coups candidats situés dans un rayon de 2 autour des pions déjà placés. Sur un plateau vide, le coup candidat est le centre du plateau. Cela réduit le facteur de branchement du Gomoku.

Pour améliorer encore l'élagage, les coups peuvent être ordonnés avant la recherche. On joue temporairement chaque coup candidat, on évalue rapidement la position obtenue, puis on trie les coups selon ce score.

Algorithme 3 : Ordonnancement des coups

Input : Plateau B , coups candidats C , joueur courant p , joueur MAX r

Output : Coups ordonnés

```

foreach  $a \in C$  do
    appliquer  $a$ 
     $score(a) \leftarrow Eval(B, r)$ 
    annuler  $a$ 
if  $p = r$  then
    | trier  $C$  par score décroissant
else
    | trier  $C$  par score croissant
return  $C$ 

```

Pour un noeud MAX, les coups les plus favorables sont donc explorés en premier. Pour un noeud MIN, ce sont les coups les plus défavorables à MAX qui sont explorés en premier. Cet ordonnancement ne change pas les règles du jeu ni les coups possibles ; il modifie seulement l'ordre de parcours de l'arbre.

7 Expériences

7.1 Expérience 1 : efficacité de l'élagage alpha-bêta

Cette première expérience a pour objectif de vérifier l'effet de l'élagage alpha-bêta sur notre algorithme de recherche. Nous voulons savoir si alpha-bêta permet de réduire le nombre de noeuds explorés sans modifier la qualité de la décision obtenue par Minimax.

Nous comparons trois variantes :

- Minimax classique ;
- Minimax avec élagage alpha-bêta, sans ordonnancement des coups ;
- Minimax avec élagage alpha-bêta et ordonnancement des coups.

Pour que la comparaison soit équitable, les paramètres sont fixés. Les trois algorithmes utilisent les mêmes positions de test, les mêmes profondeurs de recherche, le même générateur de coups candidats et la même fonction d'évaluation. Nous utilisons ici l'Évaluation B, car elle prend en compte la longueur des alignements ainsi que leurs extrémités ouvertes ou fermées. Elle est donc plus informative que l'Évaluation A, tout en restant moins coûteuse que l'Évaluation C.

L'expérience est réalisée sur cinq positions fixes représentant différentes situations de jeu : ouverture, milieu de partie équilibré, menace simple, menaces mutuelles et milieu de partie plus dense. Les profondeurs testées sont 1, 2 et 3.

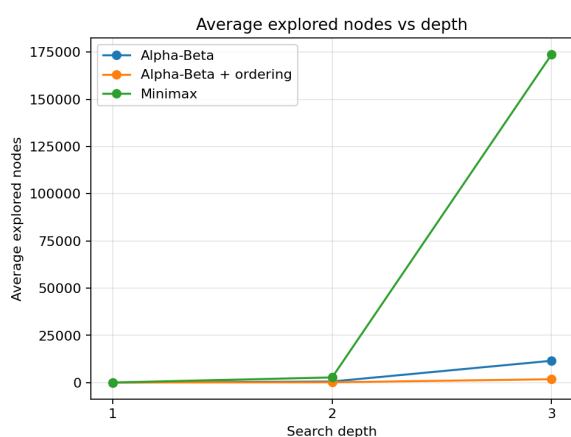
Pour chaque exécution, nous mesurons :

- le meilleur coup choisi ;
- le meilleur score obtenu ;
- le nombre de noeuds explorés ;
- le nombre de coupures alpha-bêta ;
- le temps de calcul.

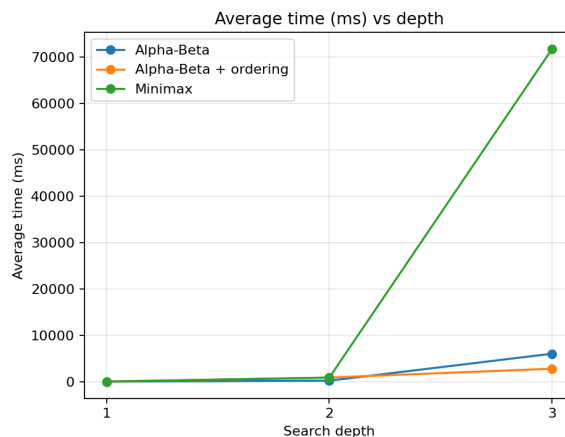
Profondeur	Algorithme	Noeuds moyens	Temps moyen (ms)	Coupures moyennes
1	Minimax	48.6	12.34	0.0
1	Alpha-Beta	48.6	13.78	0.0
1	Alpha-Beta + ordering	48.6	24.64	0.0
2	Minimax	2727.0	875.47	0.0
2	Alpha-Beta	573.6	214.87	44.8
2	Alpha-Beta + ordering	151.6	887.87	46.2
3	Minimax	173762.8	71749.55	0.0
3	Alpha-Beta	11555.6	6029.39	367.0
3	Alpha-Beta + ordering	1782.6	2795.80	127.2

TABLE 7 – Comparaison de Minimax, alpha-bêta et alpha-bêta avec ordonnancement

Les deux figures suivantes montrent les résultats les plus importants de cette expérience : l'évolution du nombre moyen de noeuds explorés et l'évolution du temps moyen de calcul selon la profondeur.



(a) Noeuds explorés



(b) Temps de calcul

FIGURE 1 – Impact de l'élagage alpha-bêta selon la profondeur

La figure 1a montre que le nombre de noeuds augmente très rapidement avec la profondeur pour Minimax. À profondeur 3, Minimax explore en moyenne 173762.8 noeuds, alors qu'alpha-bêta sans ordonnancement en explore 11555.6. Avec l'ordonnancement des coups, ce nombre descend à 1782.6. Cela correspond à une réduction d'environ 93.35% pour alpha-bêta seul, et d'environ 98.97% avec l'ordonnancement.

La figure 1b montre que le temps de calcul suit globalement la même tendance. À profondeur 3, Minimax devient très coûteux, avec un temps moyen d'environ 71749.55

ms. Alpha-bêta réduit ce temps à environ 6029.39 ms, et alpha-bêta avec ordonnancement à environ 2795.80 ms.

On remarque cependant qu'à profondeur 2, l'ordonnancement explore moins de noeuds mais prend plus de temps que l'alpha-bêta sans ordonnancement. Cela s'explique par le coût du tri des coups : chaque coup candidat est évalué avant la recherche pour choisir un meilleur ordre de parcours. À faible profondeur, ce coût supplémentaire peut être plus important que le gain obtenu par les coupures. À profondeur plus grande, le gain en nombre de noeuds devient suffisant pour compenser ce coût.

Enfin, les variantes alpha-bêta obtiennent le même meilleur score que Minimax dans tous les cas testés. Dans deux cas, le coup choisi est différent avec l'ordonnancement, mais le score reste identique. Cela signifie que plusieurs coups sont considérés comme équivalents par la fonction d'évaluation.

Cette expérience confirme donc que l'élagage alpha-bêta est efficace pour notre IA de Gomoku. Il réduit fortement le nombre de noeuds explorés sans changer la valeur obtenue par Minimax. L'ordonnancement des coups devient particulièrement utile lorsque la profondeur de recherche est suffisamment élevée.

7.2 Expérience 2 : comparaison des fonctions d'évaluation

Cette expérience compare les trois fonctions d'évaluation A, B et C dans le même cadre de recherche. Contrairement à l'expérience précédente, l'objectif n'est plus de comparer Minimax et alpha-bêta. Ici, l'algorithme de recherche est fixé : nous utilisons alpha-bêta avec ordonnancement des coups. Seule la fonction d'évaluation change.

Les trois évaluations comparées sont :

- **Évaluation A** : évaluation de base, enrichie avec des critères défensifs sur certains motifs ouverts ;
- **Évaluation B** : évaluation des motifs ouverts et bloqués ;
- **Évaluation C** : évaluation par fenêtres de cinq cases, avec une prise en compte plus globale du potentiel de la position.

Pour que la comparaison soit équitable, les mêmes conditions sont utilisées pour les trois fonctions : même taille de plateau, même générateur de coups candidats, même algorithme alpha-bêta avec ordonnancement, mêmes positions de test et mêmes profondeurs de recherche. Les profondeurs testées sont 1, 2 et 3.

L'expérience utilise six positions fixes, choisies pour tester différents types de situations : pression par longueur d'alignement, motifs ouverts ou bloqués, potentiel de fenêtres de cinq cases, équilibre entre attaque et défense, contrôle de l'espace central et milieu de partie plus complexe.

Profondeur	Évaluation	Temps (ms)	Noeuds	Coupures
1	Évaluation A	37.75	58.00	0.00
1	Évaluation B	27.29	58.00	0.00
1	Évaluation C	245.87	58.00	0.00
2	Évaluation A	1316.81	190.50	55.67
2	Évaluation B	903.72	224.00	55.17
2	Évaluation C	8244.92	209.50	55.33
3	Évaluation A	6152.15	4196.17	145.50
3	Évaluation B	4549.07	4120.17	151.00
3	Évaluation C	35588.19	4151.83	136.67

TABLE 8 – Comparaison des fonctions d'évaluation avec alpha-bêta et ordonnancement

La figure suivante est la plus représentative pour cette expérience. Elle montre l'évolution du temps moyen de calcul selon la profondeur pour les trois fonctions d'évaluation.

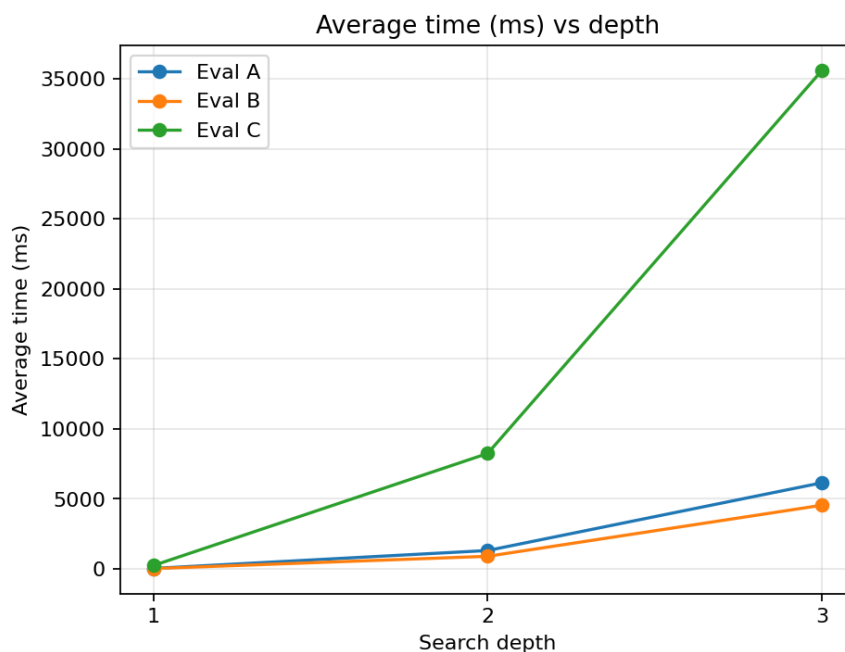


FIGURE 2 – Temps moyen de calcul des fonctions d'évaluation selon la profondeur

On observe que le temps de calcul augmente fortement avec la profondeur pour les trois fonctions. Cependant, l'Évaluation C est nettement plus coûteuse que les deux autres. À profondeur 3, elle prend en moyenne environ 35588.19 ms, alors que l'Évaluation B prend environ 4549.07 ms. L'Évaluation C est donc environ 7.8 fois plus lente que l'Évaluation B à cette profondeur.

Les nombres de noeuds explorés sont en revanche assez proches entre les trois évaluations. À profondeur 3, les trois valeurs restent autour de 4100 noeuds. Cela signifie que l'écart de temps ne vient pas principalement d'un arbre de recherche plus grand, mais plutôt du coût de calcul de chaque appel à la fonction d'évaluation.

Un résultat important est aussi que les fonctions d'évaluation ne choisissent pas toujours les mêmes coups. Sur les 18 cas position-profondeur testés, les trois évaluations donnent le même meilleur coup dans 11 cas, et des coups différents dans 7 cas. Cela

montre que les évaluations ne diffèrent pas seulement par leur coût, mais aussi par leur comportement stratégique.

L'Évaluation A n'est plus une simple évaluation minimale : elle a été modifiée pour inclure des critères défensifs supplémentaires. Cela explique pourquoi elle peut être plus lente que l'Évaluation B dans certains cas. L'Évaluation B offre le meilleur compromis dans cette expérience : elle détecte les formes ouvertes et bloquées tout en restant relativement rapide. L'Évaluation C donne une analyse plus riche du potentiel des fenêtres de cinq cases, mais son coût est trop élevé pour être utilisée systématiquement à profondeur importante.

7.3 Expérience 3 : sélection des configurations d'IA

L'expérience 3 sert à choisir les configurations d'IA les plus adaptées pour les niveaux Easy, Medium et Hard. Les résultats des expériences précédentes sont utilisés pour limiter les configurations testées.

D'après l'expérience 1, alpha-bêta avec ordonnancement des coups est plus efficace que Minimax classique. Toutes les IA candidates utilisent donc cette méthode de recherche. D'après l'expérience 2, l'Évaluation C est trop coûteuse pour être utilisée dans les configurations principales. Les candidats testés utilisent donc surtout les Évaluations A et B, avec différentes profondeurs.

Les configurations candidates sont les suivantes :

IA candidate	Évaluation	Profondeur
A1	Évaluation A	1
B1	Évaluation B	1
A2	Évaluation A	2
B2	Évaluation B	2
A3	Évaluation A	3
B3	Évaluation B	3

TABLE 9 – Configurations candidates pour la sélection des niveaux d'IA

Le protocole consiste à organiser un tournoi entre ces six IA candidates. Chaque paire d'IA joue plusieurs parties, avec échange du premier joueur. Les parties sont limitées à un nombre maximal de coups ; si aucun joueur ne gagne avant cette limite, la partie est comptée comme nulle.

Le tournoi contient 150 parties au total. Les résultats globaux sont les suivants : 55 victoires pour le joueur noir, 65 victoires pour le joueur blanc et 30 matches nuls. Les victoires des deux couleurs sont donc relativement équilibrées dans ce test.

IA	Évaluation	Profondeur	Win rate	Score rate	Temps/coup (ms)
B3	B	3	0.80	0.85	20273.71
B2	B	2	0.50	0.65	8562.26
A3	A	3	0.50	0.60	40418.54
A1	A	1	0.40	0.50	188.27
A2	A	2	0.20	0.40	12109.13
B1	B	1	0.00	0.00	83.82

TABLE 10 – Classement des IA candidates

La figure la plus utile pour cette expérience est le compromis entre force et temps de calcul. Elle permet de visualiser à la fois le score rate et le coût moyen par coup.

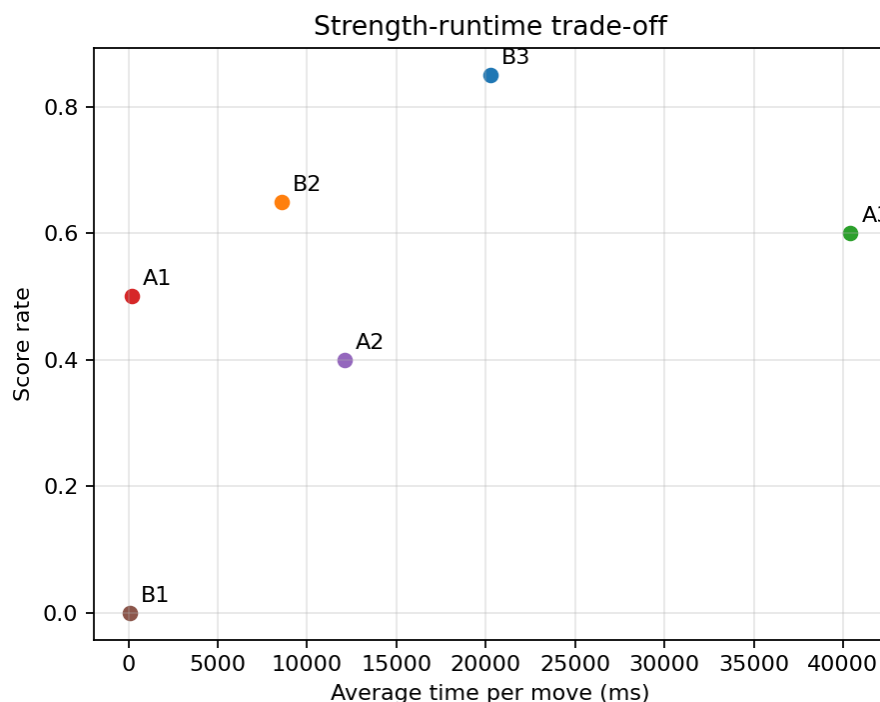


FIGURE 3 – Compromis entre performance et temps moyen par coup

Le candidat B3 obtient le meilleur score rate, avec 0.85. Il est donc le plus fort dans ce tournoi. Cependant, il reste coûteux, avec environ 20273.71 ms par coup en moyenne. Le candidat B2 obtient un score rate de 0.65, avec un temps moyen plus faible. Il représente donc un bon compromis pour un niveau intermédiaire.

Le candidat A3 obtient un score rate proche de B2, mais son temps moyen par coup est beaucoup plus élevé. Il est donc moins intéressant comme configuration principale. A1 est beaucoup plus rapide et reste suffisamment compétitif pour un niveau facile. B1 est très rapide, mais perd toutes ses parties dans cette expérience, ce qui le rend trop faible même pour le niveau Easy.

À partir de ces résultats, nous retenons la configuration finale suivante :

Difficulté	Configuration	Description
Easy	A1	Évaluation A, profondeur 1
Medium	B2	Évaluation B, profondeur 2
Hard	B3	Évaluation B, profondeur 3

TABLE 11 – Configuration finale des niveaux de difficulté

Le niveau Easy utilise A1, car il est rapide et plus solide que B1. Le niveau Medium utilise B2, car il offre un bon équilibre entre performance et temps de calcul. Le niveau Hard utilise B3, car c'est la configuration la plus forte du tournoi, même si son coût est plus élevé.

8 Tournoi entre intelligences artificielles

8.1 Protocole expérimental

Pour évaluer nos différentes intelligences artificielles, nous avons organisé un tournoi entre les trois niveaux retenus : Easy, Medium et Hard. Chaque paire d'IA s'affronte sur un ensemble de 200 parties. Afin de limiter l'influence du joueur qui commence, les deux IA échangent leur ordre de jeu : une moitié des parties est jouée avec la première IA en joueur 1, et l'autre moitié avec la seconde IA en joueur 1.

Cependant, dans un jeu comme le Gomoku, commencer systématiquement depuis un plateau vide peut produire des parties très similaires, surtout lorsque les IA utilisent des fonctions d'évaluation déterministes. Pour éviter de mesurer plusieurs fois presque la même partie, nous avons défini un ensemble fixe de positions d'ouverture. Ces positions servent de points de départ variés pour les matchs du tournoi.

Notre ensemble d'ouvertures contient dix configurations différentes, notées de O1 à O10. Elles représentent plusieurs familles de positions : regroupement central, tension décalée, équilibre diagonal, centre élargi, ainsi que leurs variantes symétriques ou avec inversion des joueurs. Chaque position contient déjà plusieurs coups placés sur le plateau, ainsi que le joueur qui doit jouer le prochain coup.

L'objectif de ces ouvertures est double. Premièrement, elles permettent de tester les IA dans des situations plus diverses qu'un simple départ depuis le plateau vide. Deuxièmement, elles rendent les résultats plus robustes, car une IA ne peut pas gagner uniquement grâce à une séquence d'ouverture particulière. Les positions choisies restent équilibrées : elles ne contiennent pas de victoire immédiate ni de ligne forcée évidente, afin que le résultat dépende principalement de la qualité de recherche et de la fonction d'évaluation de chaque IA.

Chaque ouverture est représentée par une liste de coups de la forme $(r, c, joueur)$, où r et c sont les coordonnées de la case, et où le joueur vaut 1 ou -1 . Avant chaque partie, le plateau est reconstruit à partir de cette liste. Nous avons également utilisé des transformations simples, comme la symétrie horizontale et l'inversion des joueurs, pour obtenir des positions comparables mais non identiques.

Pour chaque match, nous enregistrons :

- le nombre de victoires de chaque IA ;
- le nombre de matchs nuls ;
- le temps moyen de calcul par coup ;
- l'influence du premier joueur ;
- éventuellement le nombre moyen de coups joués.

Cette méthode permet donc de comparer les IA non seulement sur leur taux de victoire, mais aussi sur leur efficacité temporelle et leur robustesse face à différentes situations de jeu.

8.2 Résultats expérimentaux du tournoi

Afin d'analyser plus précisément les performances des trois niveaux d'IA, nous avons représenté les résultats du tournoi sous plusieurs formes complémentaires. Chaque couple d'IA a joué 200 parties, ce qui permet d'obtenir des résultats plus stables que le minimum de 50 parties demandé.

La figure 4 présente le taux de score obtenu par chaque IA contre chaque adversaire. On observe que Medium domine Easy avec un taux de score de 90 %, tandis que Hard

obtient 80 % contre Easy. Dans la confrontation directe entre Medium et Hard, Hard obtient un taux de score de 62,5 %. Ces résultats confirment donc la hiérarchie globale suivante :

$$\text{Hard} > \text{Medium} > \text{Easy}.$$

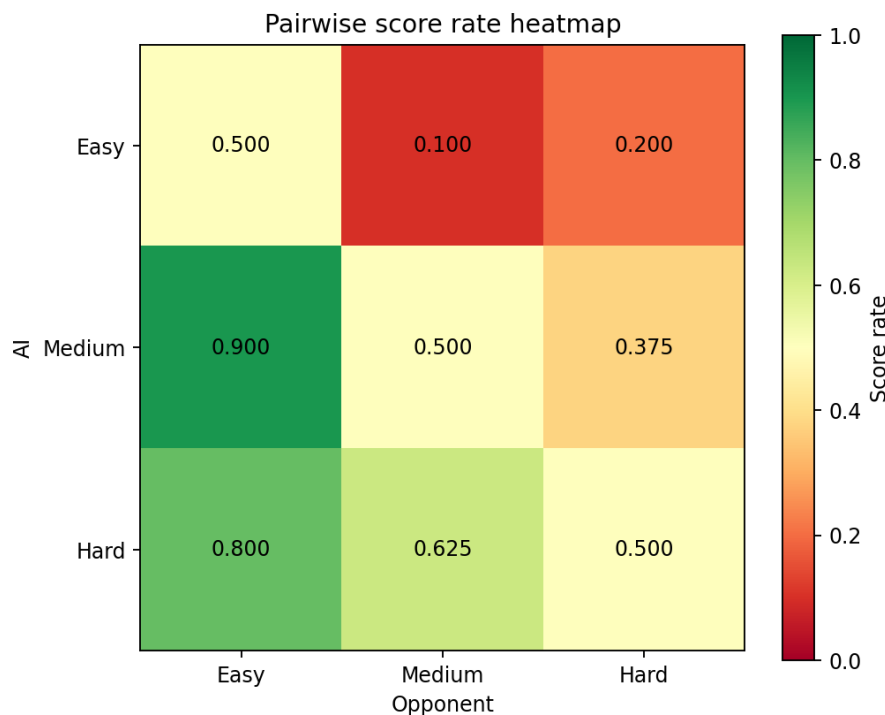


FIGURE 4 – Taux de score entre chaque paire d'IA. Chaque case indique le taux de score de l'IA en ligne contre l'IA en colonne.

On remarque cependant que Medium obtient un meilleur résultat contre Easy que Hard contre Easy. Cela ne remet pas en cause le classement final, car Hard bat directement Medium. Ce phénomène peut s'expliquer par le fait que Medium est particulièrement efficace contre le style de jeu de Easy, tandis que Hard utilise une stratégie plus générale, qui se révèle surtout supérieure dans les confrontations plus équilibrées.

La figure 5 montre l'évolution du taux de score cumulé au cours des 200 parties. Au début, les courbes fluctuent davantage, car chaque partie a une influence importante sur le score total. Ensuite, les résultats se stabilisent progressivement. Les trois courbes convergent vers leurs valeurs finales : 90 % pour Medium contre Easy, 80 % pour Hard contre Easy, et 62,5 % pour Hard contre Medium.

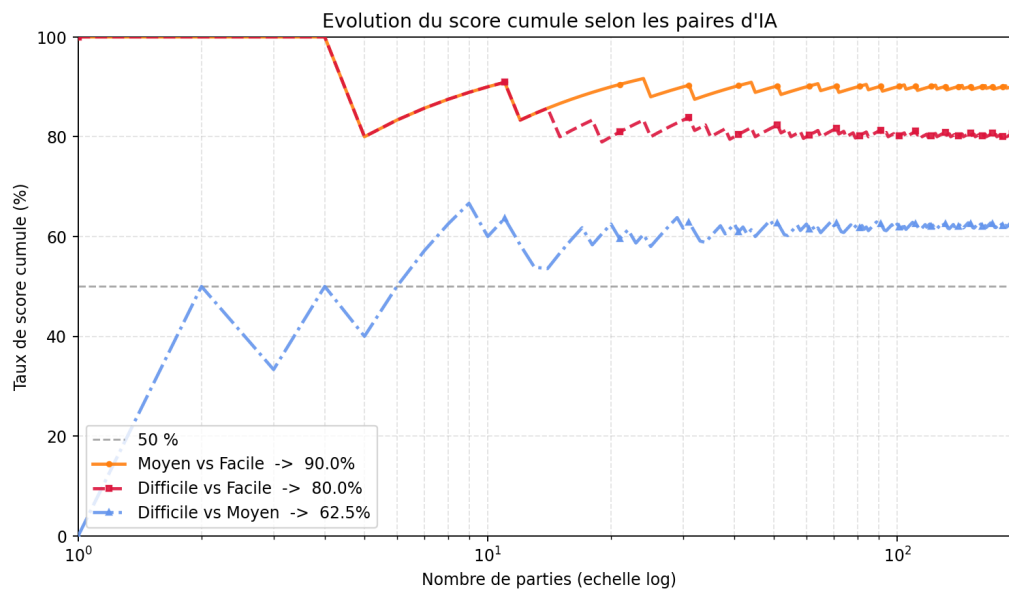


FIGURE 5 – Évolution du taux de score cumulé selon les paires d'IA. L'axe des abscisses est en échelle logarithmique afin de mieux visualiser les premières parties.

Cette stabilisation montre que les résultats ne sont pas dus uniquement à quelques parties isolées. Le choix de 200 parties par confrontation rend donc l'analyse plus robuste.

Nous avons aussi étudié l'influence du joueur qui commence. Dans notre implémentation du Gomoku, le joueur noir commence la partie. La figure 6 compare les taux de victoire obtenus avec les noirs et avec les blancs pour chaque niveau d'IA.

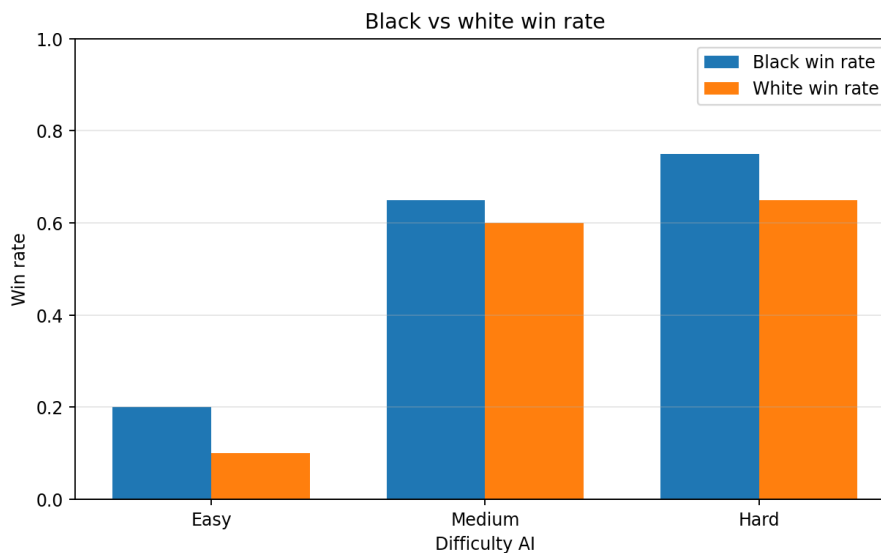


FIGURE 6 – Comparaison des taux de victoire selon la couleur jouée. Le joueur noir commence la partie.

Les résultats montrent que le taux de victoire avec les noirs est supérieur au taux de victoire avec les blancs pour les trois niveaux d'IA. Cela confirme l'existence d'un avantage du premier joueur. Cependant, ce biais est limité dans notre protocole expérimental, car les IA échangent leur ordre de jeu : chaque IA joue une partie des matchs en premier joueur et une autre partie en second joueur.

Enfin, la figure 7 compare le taux de score global des IA avec leur temps moyen de calcul par coup. Elle permet d’analyser le compromis entre force de jeu et coût temporel.

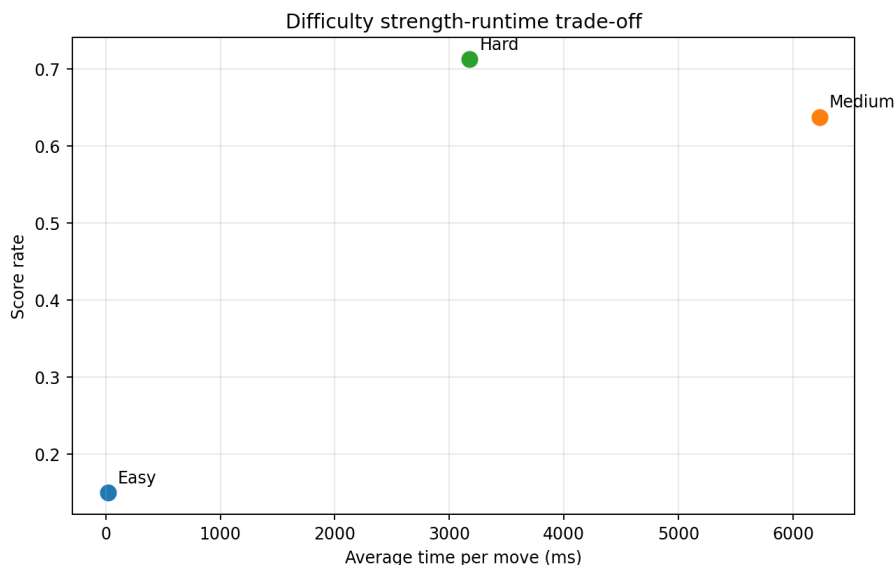


FIGURE 7 – Compromis entre la force de jeu et le temps moyen de calcul par coup.

Easy est très rapide, avec un temps moyen par coup très faible, mais son taux de score est également limité. Medium améliore fortement la qualité de jeu, mais son temps de calcul devient très élevé. Hard obtient le meilleur taux de score global tout en étant plus rapide que Medium. Il représente donc le meilleur compromis entre performance et temps de calcul.

Ces résultats justifient le choix final des trois niveaux de difficulté. Easy correspond à une IA simple et rapide, Medium à une IA plus forte mais coûteuse en temps de calcul, et Hard à une IA plus robuste, capable d’obtenir les meilleurs résultats avec un temps de calcul plus raisonnable.

8.3 Observations sur les résultats

Une observation intéressante concerne les résultats contre l’IA Easy. Medium obtient un taux de score de 90 % contre Easy, alors que Hard obtient un taux de score de 80 % contre Easy. À première vue, ce résultat peut sembler surprenant, puisque Hard est censée être plus forte que Medium.

Cependant, ce résultat ne signifie pas que Medium est globalement meilleure que Hard. Dans la confrontation directe entre Medium et Hard, Hard obtient un taux de score de 62,5 %, ce qui confirme sa supériorité globale. La différence observée contre Easy peut s’expliquer par la nature des configurations utilisées. Easy et Medium reposent sur la même fonction d’évaluation, notée EvalA, mais avec des profondeurs de recherche différentes. Medium peut donc être vue comme une version plus profonde de Easy : elle évalue les positions selon les mêmes critères, mais elle anticipe plus loin les menaces et les opportunités. Elle exploite donc particulièrement bien les faiblesses de Easy, car leurs décisions sont guidées par une logique similaire.

Hard, en revanche, utilise une autre fonction d’évaluation, EvalB. Cette fonction semble plus robuste dans les confrontations équilibrées, mais elle n’est pas nécessairement spécialisée pour exploiter les erreurs de Easy. Cela peut expliquer pourquoi son taux

de score contre Easy est légèrement inférieur à celui de Medium, malgré une meilleure performance globale.

La confrontation entre Hard et Medium est donc essentielle pour interpréter correctement les résultats. Le taux de score de 62,5 % montre que Hard est bien supérieure à Medium, mais pas de manière écrasante. Cette observation met en évidence un compromis important dans notre projet : la profondeur de recherche améliore fortement la qualité de jeu, comme le montre le passage de Easy à Medium, mais elle ne suffit pas à elle seule. La qualité de la fonction d'évaluation joue également un rôle déterminant. Une IA moins profonde mais mieux guidée par son heuristique peut obtenir de meilleurs résultats qu'une IA plus profonde mais fondée sur une évaluation moins pertinente.

Ainsi, ces résultats montrent que la force d'une IA de Gomoku dépend à la fois de sa capacité à anticiper plusieurs coups et de la pertinence des critères utilisés pour évaluer les positions. La profondeur permet de voir plus loin, tandis que la fonction d'évaluation permet de mieux choisir ce qu'il faut regarder.

9 Difficultés rencontrées

Coût de calcul élevé de Minimax

Difficulté. Même avec une génération de coups candidats, le nombre de positions explorées augmente rapidement avec la profondeur de recherche. Une recherche Minimax classique devient donc trop lente pour les niveaux les plus difficiles.

Solution. Nous avons utilisé l'élagage alpha-bêta afin de couper les branches qui ne peuvent plus influencer la décision finale. Nous avons également utilisé un ordonnancement des coups pour explorer en priorité les coups les plus prometteurs, ce qui améliore l'efficacité de l'élagage.

Évaluation A initialement trop simple

Difficulté. Lors des premiers tests, l'Évaluation A était trop simple. Elle ne prenait en compte que la longueur des alignements consécutifs. Elle était donc rapide, mais elle ne détectait pas suffisamment les menaces importantes, notamment les motifs ouverts. Les résultats obtenus étaient alors peu comparables avec ceux des Évaluations B et C.

Solution. Nous avons enrichi l'Évaluation A en ajoutant des critères supplémentaires, notamment la prise en compte des deux ouverts et des trois ouverts. Cette version reste relativement simple, mais elle devient plus défensive et plus pertinente pour les expériences. Les valeurs exactes des poids utilisés sont présentées dans la partie consacrée aux fonctions d'évaluation.

Choix instable des niveaux de difficulté

Difficulté. Le choix des trois niveaux finaux d'IA n'a pas été immédiat. Dans une première version, nous avons choisi les configurations A1, A3 et B2 pour représenter respectivement les niveaux Easy, Medium et Hard. Cependant, les résultats obtenus étaient instables : le taux de victoire de B2 contre A3 était inférieur à 50%, alors que B2 était censée représenter un niveau plus difficile. Cela a mis en évidence l'importance de la

profondeur de recherche, mais aussi ses limites lorsque la fonction d'évaluation n'est pas suffisamment adaptée.

Nous avons ensuite essayé de remplacer B2 par B3, en supposant qu'une profondeur plus grande rendrait l'IA plus fiable. Pourtant, les résultats ont montré que cette configuration n'était pas satisfaisante, et qu'elle pouvait même obtenir de moins bons résultats qu'une IA plus simple comme A1. Ce comportement montre qu'augmenter la profondeur ne garantit pas automatiquement une meilleure qualité de jeu. En effet, Minimax dépend fortement de la fonction d'évaluation utilisée aux feuilles de l'arbre de recherche. Si cette fonction sous-estime certaines menaces importantes, une recherche plus profonde peut simplement propager une mauvaise estimation sur un plus grand nombre de positions.

Dans notre cas, l'ancienne version de l'Évaluation B ne prenait pas suffisamment en compte certaines menaces combinées, comme les doubles trois ouverts, les combinaisons entre un trois ouvert et un quatre bloqué, les combinaisons entre un trois ouvert et un quatre ouvert, ou encore les doubles quatre bloqués. Ces motifs sont pourtant très importants au Gomoku, car ils peuvent créer plusieurs menaces simultanées et forcer l'adversaire à défendre dans une position défavorable.

Solution. Nous avons donc modifié l'Évaluation B afin de mieux détecter ces motifs tactiques. En plus de l'évaluation classique des alignements, nous avons ajouté des bonus spécifiques pour les configurations suivantes :

- double trois ouvert : +6000 ;
- trois ouvert + quatre bloqué : +12000 ;
- trois ouvert + quatre ouvert : +18000 ;
- double quatre bloqué : +8000 ;
- trois ouvert + trois endormi : +1200.

Ces ajouts permettent à l'IA de mieux reconnaître les positions qui créent plusieurs menaces en même temps. Après cette modification, l'Évaluation B est devenue plus cohérente avec l'augmentation de profondeur : une recherche plus profonde apporte alors réellement une meilleure anticipation, au lieu d'amplifier les défauts de l'heuristique. Le choix final des niveaux de difficulté a donc été fait en tenant compte à la fois du taux de victoire, du temps moyen de calcul et de la stabilité des résultats entre les différentes configurations.

10 Conclusion

Ce projet nous a permis d'implémenter un moteur de Gomoku et plusieurs intelligences artificielles basées sur Minimax, l'élagage alpha-bêta et des fonctions d'évaluation.

Les expériences montrent que l'élagage alpha-bêta réduit fortement le nombre de noeuds explorés, surtout avec un bon ordonnancement des coups. Elles montrent aussi que la fonction d'évaluation joue un rôle important : une heuristique plus riche peut améliorer les décisions, mais elle augmente souvent le temps de calcul.

Les expériences intermédiaires nous ont aidés à sélectionner des configurations candidates. Cependant, le choix final des niveaux **Easy**, **Medium** et **Hard** est validé par le tournoi entre IA, qui compare directement les configurations retenues dans des parties complètes.

Le tournoi final confirme une hiérarchie globale entre les niveaux, tout en montrant qu'une IA plus forte n'est pas seulement une IA plus profonde. La performance dépend à la fois de la profondeur de recherche, de la qualité de la fonction d'évaluation et du temps de calcul nécessaire pour choisir un coup.

11 Utilisation de l'IA générative

Nous avons utilisé des outils d'IA générative pendant le projet, principalement ChatGPT et Codex. Ces outils ont servi d'assistance, mais les choix finaux, les tests et l'analyse des résultats ont été vérifiés par les membres du binôme.

ChatGPT a surtout été utilisé pour la partie réflexion et rédaction. Il nous a aidés à mieux comprendre certains algorithmes, comme Minimax et l'élagage alpha-bêta, ainsi qu'à discuter les choix possibles pour les fonctions d'évaluation du Gomoku. Nous l'avons aussi utilisé pour proposer des plans d'expériences, analyser les résultats obtenus et améliorer la formulation du rapport, notamment pour la correction de certaines phrases et la structuration du texte.

Codex a été utilisé davantage pour le code et les expériences. Il nous a aidés à réorganiser certains fichiers, à nettoyer du code inutile, à rendre le projet plus modulaire et à automatiser les tests expérimentaux. Il a notamment servi à générer et exécuter des scripts d'expérience, collecter les résultats, produire des fichiers CSV, générer des graphiques et rédiger des rapports expérimentaux intermédiaires.

Nous avons également utilisé ChatGPT pour préparer des prompts plus précises à donner à Codex, par exemple pour lancer des expériences complètes avec des conditions contrôlées. Cela nous a permis de mieux préciser les fichiers à lire, les paramètres à fixer et les résultats attendus.

Les réponses générées n'ont pas été utilisées sans vérification. Le code proposé a été relu et testé, les résultats expérimentaux ont été contrôlés à partir des fichiers produits, et les explications ont été adaptées à notre implémentation réelle. L'IA générative a donc été utile pour gagner du temps et organiser le travail, mais elle n'a pas remplacé la validation humaine.

A Annexes

Les annexes peuvent contenir :

- les commandes d'exécution du programme ;
- les scripts de génération des graphiques ;
- les résultats expérimentaux complets ;
- des exemples de positions de test ;
- des extraits de code importants.